
UNIT 5 DATABASE INTEGRITY, FUNCTIONAL DEPENDENCY AND NORMALISATION

Structure	Page Nos.
5.0 Introduction	
5.1 Objectives	
5.2 Database Integrity	
5.2.1 The Keys	
5.2.2 Referential Integrity	
5.2.3 Entity Integrity	
5.3 Redundancy and Associated Problems	
5.4 Functional Dependencies	
5.5 Normalisation Using Functional Dependencies	
5.5.1 The First Normal Form	
5.5.2 The Second Normal Form	
5.5.3 The Third Normal Form	
5.5.4 Boyce Codd Normal Form	
5.6 Desirable Properties of Decomposition	
5.7 Rules of Data Normalisation	
5.7.1 Eliminate Repeating Groups	
5.7.2 Eliminate Redundant Data	
5.7.3 Eliminate Columns Not Dependent on Key	
5.8 Summary	
5.9 Answers/Solutions	

5.0 INTRODUCTION

The first block of this course discusses the database concept, relational algebra, Entity relationship model and integrity constraints in the context of relational database design. One of the key aspects of database design is to create relations without any data redundancy.

This unit first explains the concept of entity and referential integrity. It also explains the anomalies in a relational database system. To remove anomalies is through decomposing the database, you need to decompose relations into smaller relations, which are free of those anomalies. This decomposition may be lossless and dependency preserving. The Unit explains the concept of functional dependency, which is the basis of lossless decomposition of a relation into smaller relations.

The unit deals with the standard form of database relations and discusses the process of decomposing relations into different normal forms up to BCNF. The higher normal forms are discussed in the next unit.

5.1 OBJECTIVES

After going through this unit, you should be able to:

- Explain entity integrity and referential integrity constraints of a relation;
- Explain various anomalies that exist in a database system;
- Define the desirable properties of decomposing a relation;
- Define and use functional dependencies to normalise databases.

5.2 DATABASE INTEGRITY

Database integrity refers to maintaining the consistency of data in the database. Data integrity relates to the correctness of data and often is implemented using constraints on attributes in one or more relations. In this section, we will discuss more about two important integrity constraints of a database: the entity integrity constraint and the referential integrity constraint. To define these two, let us once again define the term Key with respect to a Database Management System.

5.2.1 The Keys

Candidate Key: In a relation R, a candidate key for R is a subset of the set of attributes of R, which have the following two properties:

- | | | |
|-----|--------------------|---|
| (1) | <i>Uniqueness</i> | No two distinct tuples in R have the same value for the candidate key |
| (2) | <i>Irreducible</i> | No proper subset of the candidate key has the uniqueness property. |

Every relation must have at least one candidate key which cannot be reduced further. Duplicate tuples are not allowed in relations. Any candidate key can be a composite key also. For Example, (student-id + course-id) together can form the candidate key of a relation called *Result* (student-id, course-id, marks).

Let us summarise the properties of a candidate key.

- A candidate key must be unique and irreducible.
- A candidate may involve one or more than one attribute. A candidate key that involves more than one attribute is said to be composite.

But why are we interested in candidate keys?

Candidate keys are important because they provide the basic **tuple-level identification** mechanism in a relational system. For example, if the enrolment number is the candidate key of a STUDENT relation, then the answer of the query: “Find student details from the STUDENT relation having enrolment number A0123” will output at most one tuple.

Primary Key

The primary key is the candidate key that is chosen by the database designer as the principal means of identifying entities within an entity set. The remaining candidate keys, if any, are called **alternate keys**.

Foreign Keys

Let us first give you the basic definition of foreign key.

Let R2 be a relation, then a foreign key in R2 is a subset of the set of attributes of R2, such that:

1. There exists a relation R1 (R1 and R2 not necessarily distinct) with a candidate key, and
2. For all time, each value of a foreign key in the current state or instance of R2 is identical to the value of Candidate Key in some tuple in the current state of R1.

The definition above seems to be very heavy. Therefore, let us define it in more practical terms with the help of the following example.

Example 1: Assume that in an organisation, an employee may perform different roles in different projects. Say, XYZ is coding in one project and designing in another. Assume that the information is represented by the organisation in three different relations named EMPLOYEE, PROJECT and ROLE. The ROLE relation describes the different roles required in any project.

Assume that the relational schema for the above three relations are:

EMPLOYEE (EMPID, Name, Designation)
PROJECT (PROJID, Proj_Name, Details)
ROLE (ROLEID, Role_description)

In the relations above EMPID, PROJID and ROLEID are unique and not NULL. As you can clearly observe, you can identify the complete instance of the entity set employee through the attribute EMPID. Thus, EMPID is the primary key of the relation EMPLOYEE. Similarly, PROJID and ROLEID are the primary keys for the relations PROJECT and ROLE respectively.

Let ASSIGNMENT is a relationship between entities EMPLOYEE and PROJECT and ROLE, describing which employee is working on which project and what the role of the employee is in the respective project. Figure 5.1 shows part of E-R diagram for these entities and relationships (for simplicity, no attribute has been shown).

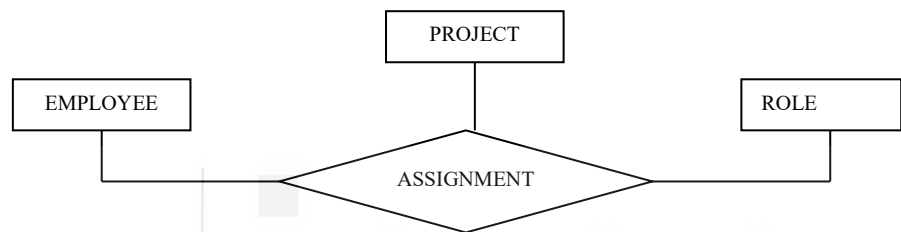


Figure 5.1: E-R diagram for employee role in development

Let us consider sample relation instances as:

EMPLOYEE

EMPID	Name	Designation
101	XYZ	Analyst
102	ABC	Receptionist
103	ARVIND	Manager

PROJECT

PROJID	Proj_name	Details
TCS	Traffic Control System	For traffic shaping.
LG	Load Generator	To simulate load for input in TCS.
B++1	B++ TREE	ISS/R turbo sys

ROLE

ROLEID	Role_description
1000	Design
2000	Coding
3000	Marketing

ASSIGNMENT

EMPID	PROJID	ROLEID
101	TCS	1000
101	LG	2000
102	B++1	3000

Figure 5.2: An example relation

You can define the relational scheme for the relation ASSIGNMENT as follows:

ASSIGNMENT (EMPID, PROJID, ROLEID)

Please note now that in the relation ASSIGNMENT (as per the definition of foreign key to be taken as R2) EMPID is the foreign key in ASSIGNMENT relation; it references the relation EMPLOYEE (as per the definition to be taken as R1) where EMPID is the primary key. Similarly, PROJID and ROLEID in the relation ASSIGNMENT are **foreign keys** referencing the relation PROJECT and ROLE respectively.

Now after defining the concept of foreign key, we can proceed to discuss the actual integrity constraints namely Referential Integrity and Entity Integrity.

5.2.2 Referential Integrity

It can be simply defined as:

The database must not contain any unmatched foreign key values.

The term “unmatched foreign key value” means a foreign key value for which there does not exist a **matching value** of the relevant candidate key in the relevant target (referenced) relation. For example, any value existing in the EMPID attribute in ASSIGNMENT relation must exist in the EMPLOYEE relation. That is, the only EMPIDs that can exist in the EMPLOYEE relation are 101, 102 and 103 for the present state/ instance of the database given in *Figure 5.2*. If we want to add a tuple with EMPID value 104 in the ASSIGNMENT relation, it will cause violation of referential integrity constraint. Logically it is obvious, after all the employee 104 does not exist, so how can s/he be assigned any work.

Database modifications can cause violations of referential integrity. We list here the referential action that you may specify for each type of database modification to preserve the referential-integrity constraint:

Delete

During the deletion of a tuple two cases can occur:

Deletion of tuple in relation having the foreign key: In such a case simply delete the desired tuple. For example, in ASSIGNMENT relation you can easily delete the first tuple.

Deletion of the target of a foreign key reference: For example, an attempt to delete an employee tuple in EMPLOYEE relation whose EMPID is 101. This employee appears not only in the EMPLOYEE but also in the ASSIGNMENT relation. Can this tuple be deleted? If you delete the tuple in EMPLOYEE relation then two unmatched tuples are left in the ASSIGNMENT relation, thus causing violation of referential integrity constraint. Thus, the following two choices exist for such deletion:

RESTRICT – The delete operation is “restricted” to only the case where there are no matching tuples in the referencing relation. For example, you can delete the EMPLOYEE record of EMPID 103 as no matching tuple in ASSIGNMENT but not the record of EMPID 101.

CASCADE – The delete operation “cascades” to delete those matching tuples also. For example, if the delete mode is CASCADE, then deleting an employee data having EMPID as 101 from EMPLOYEE relation will also cause deletion of 2 more tuples from ASSIGNMENT relation.

Insert

The insertion of a tuple in the target of reference does not cause any violation. However, insertion of a tuple in the relation in which we have the foreign key, for example, in ASSIGNMENT relation it needs to be ensured that all matching target candidate key exist; otherwise, the insert operation can be rejected. For example, one of the possible ASSIGNMENT insert operations would be (103, LG, 3000).

Modify

Modifying or updating operations changes the existing values. If these operations change the value that is the foreign key also, the only check required is the same as that of the Insert operation.

What should happen to an attempt to update a candidate key that is the target of a foreign key reference? For example, an attempt to update the PROJID “LG” for which there exists at least one matching ASSIGNMENT tuple? In general, there are the same possibilities as for DELETE operation:

RESTRICT: The update operation is “restricted” to the case where there are no matching ASSIGNMENT tuples. (It is rejected otherwise).

CASCADE – The update operation “cascades” to update the foreign key in those matching ASSIGNMENT tuples also.

5.2.3 Entity Integrity

Before describing the second type of integrity constraint, viz., Entity Integrity, you should be familiar with the concept of **NULL**.

Basically, NULL is intended as a basis for dealing with the problem of missing information. This kind of situation is frequently encountered in the real world. For example, historical records sometimes have entries such as “Date of birth unknown”. Hence it is necessary to have some way of dealing with such situations in database systems. Codd proposed an approach to this issue that makes use of special markers called NULL to represent such missing information.

A given attribute in the relation might or might not be allowed to contain NULL. But can the Primary key or any of its components (in case primary key is a composite key) contain a NULL? To answer this question an **Entity Integrity Rule** states: **No component of the primary key of a relation is allowed to accept NULL.** In other words, the definition of every attribute involved in the primary key of any basic relation must explicitly or implicitly include the specifications of NULL NOT ALLOWED.

Foreign Keys and NULL

Let us consider the relations:

DEPT		
DEPT ID	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M
D3	Research	5M

EMP			
EMP ID	ENAME	DEPT ID	SALARY
E1	Rohan	D1	40K
E2	Aparna	D1	42K
E3	Ankit	D2	30K
E4	Sangeeta		35K

Suppose that Sangeeta is not assigned any Department. In the EMP tuple corresponding to Sangeeta, therefore, there is no genuine department number that can serve as the appropriate value for the DEPTID foreign key. Thus, one cannot determine DNAME and BUDGET for Sangeeta’s department as those values are NULL. This may be a real situation where the person has newly joined and is undergoing training and will be allocated to a department only on completion of the training. Thus, NULL in foreign key values may not be a logical error.

So, the foreign key definition may be redefined to include NULL as an acceptable value in the foreign key for which there is no need to find a matching tuple.

Check Your Progress 1

Consider the following relations:

S		
SNO	SNAME	CITY
S1	Smita	Delhi
S2	Jim	Pune
S3	Ballav	Pune

P			
PNO	PNAME	COLOUR	CITY
P1	Nut	Red	Delhi
P2	Bolt	Blue	Pune
P3	Part1	White	Mumbai

S4	Seema	Delhi
S5	Salim	Agra

P4	Part2	Blue	Delhi
P5	Camera	Brown	Pune
P6	Part3	Grey	Delhi

Database Integrity and Normalisation

J			SPJ			
<u>JNO</u>	JNAME	CITY	<u>SNO</u>	<u>PNO</u>	<u>JNO</u>	QUANTITY
J1	Sorter	Pune	S1	P1	J1	200
J2	Display	Bombay	S1	P1	J4	700
J3	OCR	Agra	S2	P3	J2	400
J4	Console	Agra	S2	P2	J7	200
J5	RAID	Delhi	S2	P3	J3	500
J6	EDP	Udaipur	S3	P3	J5	400
J7	Tape	Delhi	S4	P4	J3	900

- 1) For each of the relations, as given above, list the candidate keys. Also, identify the Primary key to each of the relations.

.....

.....

.....

.....

- 2) List the entity integrity constraints, which can be found in relations S, P, J, SPJ? List the domain constraints, if any.

.....

.....

.....

.....

- 3) Which of the relations S, P, J, SPJ has referential constraints? List those constraints.

.....

.....

.....

.....

- 4) For the referential constraints as identified in question 3, suggest suitable referential actions.

.....

.....

.....

.....

5.3 REDUNDANCY AND ASSOCIATED PROBLEMS

Let us consider the following relation STUDENT.

Enrolment Number (StEnroNo)	Student Name (StName)	Student Address (StAddress)	Course Number (CoNo)	Course Name (CoName)	Instructor (CoInstructor)	Office number of the Instructor (InOffice)
050112345	Rohan	D-27, Main Road, Ranchi	MCS- 201	Problem Solution	Nayan Kumar	102
050112345	Rohan	D-27, Main Road, Ranchi	MCS- 202	Computer Organisation	Anurag Sharma	105
050112345	Rohan	D-27, Main Road, Ranchi	MCS- 203	OS	Preeti Anand	103
050111341	Aparna	B-III, Gurgaon	MCS- 203	OS	Preeti Anand	103

Figure 5.3: A state of STUDENT relation

The above relation satisfies the properties of a relation and contains a single value in each cell. Conceptually it is convenient to have all the information in one relation, as a single query to the database may produce complete information about a person. Does the student relation, as given in Figure 5.3 has any undesirable characteristics?

You may observe that Figure 5.3 contains duplicate information in several attributes. For example, the student, whose enrolment number is 050112345 has a name - Rohan and the student stays at an address “D-27, Main Road, Ranchi”. This information is repetitive in the first three attributes of tuples 1, 2 and 3 (shown in Figure 5.3 in red colour). Similarly, the information that course name for number MCS-203 is OS and it is taught by “Preeti Anand”, whose office number is 103 (shown in Figure 5.3 in purple colour). You can observe that even this information is repetitive in tuple 3 and tuple 4. Thus, the relation of Figure 5.3 has the undesirable **Data Redundancy**.

In addition, when a new student takes admission and enrolls for the course MCS-203, then the entire information about the MCS-203 will be added to the relation. Such repetitive information not only increases the size of the database, but also results in several problems, which are discussed below.

1. *Update Anomaly*: Consider the data redundancy of student name and address, as shown in Figure 5.3. Assume that the student Rohan shifts from Ranchi to New Delhi at a new address “F-102, Maidan Garhi, New Delhi”. This will require updating the address of Rohan in all the three tuples of the Student relation. All the three tuples must be updated consistently. If this does not happen, then the address of Rohan will be inconsistent. This inconsistency is the result of update operation on redundant data. Thus, this anomaly is termed an update anomaly, which causes **data inconsistency**.

2. *Insertion Anomaly*: The note that the primary key of the Student relation, as given in Figure 5.3, is composite key consisting of enrolment number and Course Number. Any new tuple to be inserted in the relation must have a value for both the attributes of the primary key, as entity integrity constraint requires that a key may not be totally or partially NULL. However, in the given relation if you want to insert the course number and course name of a new course in the database, it would not be possible until a student enrolls in that course. Similarly, information about a new student cannot be inserted in the database until the student enrolls in a course. These problems are called insertion anomalies.

3. *Deletion Anomalies*: In some instances, useful information may be lost when a tuple is deleted. For example, if you delete the tuple corresponding to student 050111341 enrolled for MCS-203, you will lose relevant information about the

student viz. enrolment number, name and address of this student. Similarly, deletion of tuple having Student Name “Rohan” and Course Number ‘MCS-202’ will result in loss of information that MCS-202 is named “Computer Organisation” having an instructor “Anurag Sharma”, whose office number is 105. This is called deletion anomaly.

The anomalies arise primarily because the relation STUDENT has information about students as well as courses. One solution to the problems is to decompose the relation into two or more smaller relations, so that a relation contains information about one thing. But what should be the basis of this decomposition? To answer the questions let us try to articulate dependence of data within a relation with the help of the following Figure 5.4:

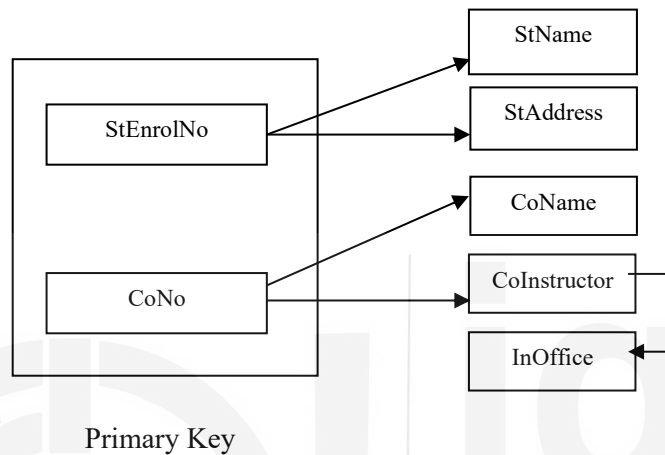


Figure 5.4: The dependencies of relation in Figure 5.3

In Figure 5.4, an arrow shows dependence of data attributes. For example, you may notice that two arrows (links) are originating from an attribute student enrolment number (*StEnrolNo*). One of these links is to student name attribute (*StName*) and the second link to student address (*StAddress*). The link between attribute *StEnrolNo* and student name attribute (*StName*) denotes that enrolment number attribute uniquely determines the student’s name. For example, in Figure 5.3 the enrolment number 050112345 uniquely determines that name of the student is Rohan. Likewise, you may determine the dependence among different attributes. You may notice that the dependence of data can exist even among the attributes which are not the part of the primary key, such as, a dependence from course instructor (*CoInstructor*) attribute to instructor office number (*InOffice*) attribute. The dependence among the attributes forms the basis of decomposition of a relation into two or more relations, this is called the normalisation. The objective of this decomposition is to reduce the three anomalies in the decomposed relations. However, normalisation should not result in loss of information, which means that you should be able to obtain the original relation’s information by taking JOIN of the relations obtained after decomposition.

A relation that needs to be normalised may have a very large number of attributes. In such relations, it is almost impossible for a person to conceptualise all the information and suggest a suitable decomposition to overcome the problems. Such relations need an algorithmic approach for finding if there are problems in a proposed database design and how to eliminate them if they exist. The discussions of these algorithms are beyond the scope of this Unit, but we will first introduce you to the basic concept that supports the process of Normalisation of large databases. So let us first define the concept of functional dependence in the subsequent section and follow it up with the concepts of normalisation.

5.4 FUNCTIONAL DEPENDENCIES

A database is a collection of related information and it is therefore inevitable that some items of information in the database would depend on some other items of information. The information is either single-valued or multi-valued. The enrolment number of a student and his/her date of birth are single-valued information; qualifications of a person or subjects that an instructor teaches are multi-valued facts. In this section, we will deal with single-valued facts, which forms the basis of the concept of functional dependency. Let us define this concept logically.

Functional Dependency (FD)

Let us consider a single universal relation schema “A”. A functional dependency denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subset of universal relation “A” specifies a constraint on the possible tuples that can form a relational state of “A”. Consider any two tuples of a relation A, say **t1** and **t2**, a FD is said to exist between two sets of attributes X to Y, if the following holds in A:

If $t1(X) = t2(X)$, then $t1(Y) = t2(Y)$ must be true.

It means that, if tuple **t1** and tuple **t2** have same values for attributes X, then to hold $X \rightarrow Y$ on “A”, **t1** and **t2** must have same values for attributes Y also.

Thus, FD $X \rightarrow Y$ means that the values of the Y component of a tuple in “A” depend on or is determined by the values of X component. In other words, the value of the Y component is uniquely determined by the value of the X component. This is functional dependency from X to Y (**but not Y to X**) that is, Y is functionally dependent on X.

The relation schema “A” determines the function dependency of Y on X ($X \rightarrow Y$) when and only when:

- 1) if tuples in “A”, which agree on their X value, then
- 2) they **must** agree on their Y values too.

Please note that if $X \rightarrow Y$ in “A”, does not mean $Y \rightarrow X$ in “A”.

Please note that functional dependency (FD) does not imply a one-to-one relationship between X and Y.

For example, the functional dependencies of Figure 5.4 are:

StEnrolNo	$\rightarrow$	StName, StAddress
CoNo	$\rightarrow$	CoName, CoInstructor
CoInstructor	$\rightarrow$	InOffice

These functional dependencies imply that there can be only one student name for each **StEnrolNo**, only one address for each student and only one course name for each **CoNo**. Please note, given this set of FDs, it is possible that two or more students, who have different enrolment numbers may have the same name. In addition, two or more students can reside at the same address. However, two different students cannot have the same enrolment number.

Similarly, consider **CoNo $\rightarrow$ CoInstructor**, the dependency implies that no subject can have more than one instructor (perhaps this is not a very realistic assumption). Functional dependencies therefore place constraints on what information the database may store. In addition, in the example above, you may be wondering if the following FDs hold:

StName $\rightarrow$ StEnrolNo (1)

Certainly, there is nothing in the given instance of the database relation presented that contradicts the functional dependencies as above. However, whether these FDs hold or not would depend on whether the university or college, whose database you are considering, allows two different students to have the same name and two different courses to have the same course names. If it was the enterprise policy to have unique course names, then (2) holds. If two students have exactly the same name, then (1) does not hold.

Let us use an E-R diagram to show various FDs (Please refer to *Figure 5.5*). A simple example of functional dependency in this ERD is when X is a primary key of an entity (e.g., enrolment number) and Y is some single-valued property or attribute of the entity (e.g., student name). $X \rightarrow Y$ then must always hold. (Why?)

Functional dependencies also arise in relationships. Let C and D be the primary keys of two strong entity sets participating in a relationship. If the relationship is one-to-one, both the FDs $C \rightarrow D$ and $D \rightarrow C$ will hold. If the relationship is many-to-one (C on many side), an FD $C \rightarrow D$ will hold but the FD $D \rightarrow C$ will NOT hold. In case, a relationship cardinality is many-to-many, then the key attributes of the participating entities would not have any functional dependency between them.

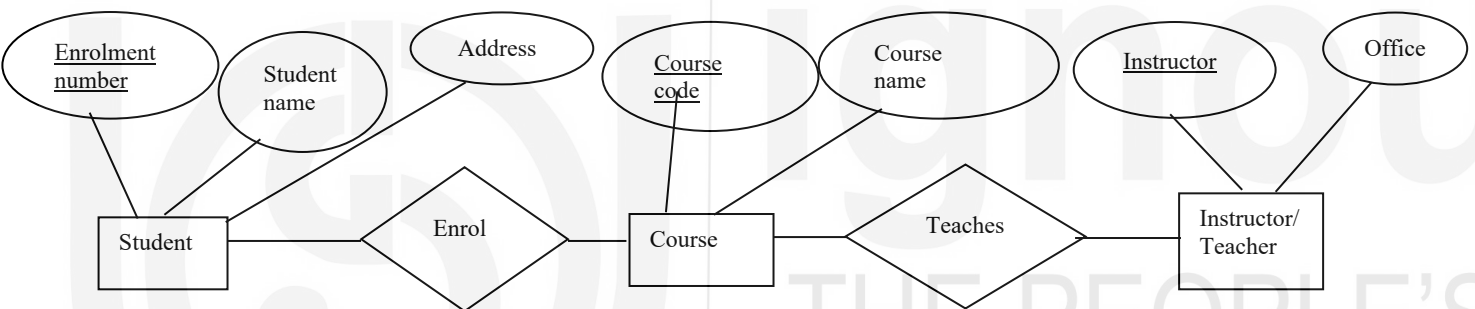


Figure 5.5: ERD of Entities – Student, Course and Teacher

ER diagram Figure 5.5 can be converted to the following relations:

Relations of Entities

STUDENT (StEnrolNo, StName, StAddress)

FDs: StEnrolNo $\rightarrow$ StName, StAddress

COURSE (CoNo, CoName)

FD: CoNo $\rightarrow$ CoName

INSTRUCTOR (CoInstructor, InOffice) //CoInstructor is instructor Id

FD: inId $\rightarrow$ InOffice

Relations of Relationships

ENROL (StEnrolNo, CoCode) (assuming many-to-many cardinality)

FD: StEnrolNo, CoNo $\rightarrow$ NULL

Assuming that one course can be taught by only one teacher, but one teacher can teach many courses, you need to redesign COURSE relation as:

COURSE (CoNo, CoName, CoInstructor)

FDs: CoNo $\rightarrow$ CoName, CoInstructor

Identification of FDs:

Identification of FDs, in general, is not trivial. You need to study the domain of attributes and relationships among these attributes. You cannot identify FDs by using a set of rules. The following example explains this.

Consider the following relation:

STUDENT-COURSE (enrolno, sname, cname, classlocation, hours)

The following functional dependencies may exist on this relation (you should identify the FDs primarily from constraints, there is no thumb rule to do so otherwise):

- **enrolno** → **sname** (the enrolment number of a student uniquely determines the student names alternatively; you can say that **sname** is functionally determined/dependent on enrolment number).
- **classcode** → **cname, classlocation**, (the value of a class code uniquely determines the class name and class location).
- **enrolno, classcode** → **Hours** (a combination of enrolment number and class code values uniquely determines the number of hours and students' study in the class per week (Hours)).

The semantic property of functional dependency explains how the attributes in "A" are related to one another. A FD in "A" must be used to specify constraints on its attributes that must hold at all times.

For example, a FD (State, City, Place) → Pin_code should hold for any address in India. It is also possible that certain functional dependencies may cease to exist in the real world if the relationship changes, for example, the FD Pin_code → Area_code used to exist as a relationship between postal codes and telephone number area codes in India, however. with the proliferation of mobile telephone, the FD is no longer true.

The set of FDs over a relation can be optimised to obtain a minimal set of FDs called the canonical cover. However, these topics are beyond the scope of this course and can be studied by consulting further readings.

5.5 NORMALISATION USING FUNCTIONAL DEPENDENCIES

Codd in the year 1972 presented three normal forms (1NF, 2NF, and 3NF). These were based on functional dependencies among the attributes of a relation. Later Boyce and Codd proposed another normal form called the Boyce-Codd normal form (BCNF). The fourth and fifth normal forms are based on multi-valued dependency and join dependencies and were proposed later. In this section we will cover normal forms till BCNF only. Fourth and fifth normal forms are discussed in the next unit. For all practical purposes, 3NF or the BCNF are quite adequate since they remove the anomalies discussed for most common situations. It should be clearly understood that there is no obligation to normalise relations to the highest possible level. Performance should be taken into account and sometimes an organisation may take a decision not to normalise, say, beyond third normal form. But it should be noted that such designs should be careful enough to take care of anomalies that would result because of the decision above.

Intuitively, the second and third normal forms are designed to result in relations such that each relation contains information about only one thing (either an entity or a relationship). A sound E-R model of the database would ensure that all relations either provide facts about an entity or about a relationship resulting in the relations that are obtained being in 2NF or 3NF.

The normalisation of a relation till BCNF is established using the FDs. The normalisation process depends on the assumptions that:

- 1) a set of functional dependencies is given for each relation, and
- 2) Each relation has a designated primary key.

The normalisation process proceeds in a top-down fashion by evaluating each relation against the criteria for normal forms and decomposing relations as found necessary during analysis. Therefore, normalisation upto BCNF is looked upon as a process of analysing the given relation schemas based on their condition (FDs and Primary Keys) to achieve the desirable properties:

- firstly, minimising redundancy, and
- secondly minimising the insertion, deletion and update anomalies.

Thus, the normalisation provides the database designer with:

- 1) a formal framework for analysing relation schemas.
- 2) a series of normal form tests that can be normalised to any desired degree.

Decomposition through normalisation should include any or both of the following properties.

- 1) the lossless join and non-additive join property, and
- 2) the dependency preservation property.

Do you always normalise the database to the highest normal form? Due to performance reasons, database designers sometimes leave relations in lower normalisation forms. This is called denormalisation.

Let us now define the normal forms in more detail.

5.5.1 The First Normal Form (1NF)

Let us first define 1NF:

Definition: A relation (table) R is in 1NF if every attribute of R takes atomic values. In other words, the following conditions hold in R :

1. *There are no duplicate rows or tuples in the relation.*
2. *Each data value stored in the relation is single-valued.*
3. *Entries in a column (attribute) are of the same kind (type).*

Please note that in a 1NF relation, the order of the tuples (rows) and attributes (columns) does not matter.

The first requirement above means that the relation **must have a key**. The key may be single attribute or composite key. It may even, possibly, contain all the columns. The first normal form defines only the basic structure of the relation and does not resolve the anomalies discussed in Section 5.3.

The relation STUDENT (StEnrolNo, StName, StAddress, CoNo, CoName, CoInstructor, InOffice) of Figure 5.3 is in 1NF. The primary key of the relation is a composite key of attributes StEnrolNo and CoNo.

5.5.2 The Second Normal Form (2NF)

The relation of Figure 5.3 is in 1NF, yet it suffers from all the anomalies. Therefore, a second normal form may be defined to overcome the anomalies.

Definition: A relation is in Second Normal Form (2NF) if it fulfills the following criteria (assuming the relation has only one candidate key, which is selected as the primary key of the relation):

- (i) *The relation fulfills the criteria of the First Normal Form (1NF), and*
- (ii) *All those attributes, which are not part of any primary key, are fully functionally dependent on the primary keys.*

Key features of 2NF:

- The partial dependency will exist, only if the primary key is composite. Therefore, if the primary key of a relation consists of a single attribute, then the given 1NF relation would be in 2NF also.
- This normal form decomposes a relation so that relations store information about one thing.

Please refer to *Figure 5.4*, which illustrates the FDs in the relation STUDENT (StEnrolNo, StName, StAddress, CoNo, CoName, CoInstructor, InOffice). The FDs, as shown in *Figure 5.4* are:

StEnrolNo	→	StName, StAddress	(1)
CoNo	→	CoName, CoInstructor	(2)
CoInstructor	→	InOffice	(3)

Since, from the above FDs, you can create a FD:

StEnrolNo, CoNo → StName, StAddress, CoName, CoInstructor, InOffice

Therefore, the attributes StEnrolNo and CoNo together form the composite key to the relation STUDENT. The relation has only one candidate key, therefore, all the attributes, other than StEnrolNo and CoNo, of the relation are not part of a candidate key. These attributes are also called non-key attributes. The relation STUDENT is in 1NF, but is it in 2NF? No, the FDs at (1) and (2) are clearly violating the fully functionally dependent criteria of 2NF (*Figure 5.4*). Therefore, the relation suffers from the anomalies as shown in *Figure 5.3*. Next, how will you decompose the STUDENT relation to 2NF relations? You may use FDs of (1), (2) and (3) to do so. FD (1) can be used to create a 2NF relation consisting of these three attributes from STUDENT relation. This part new relation is:

STUDENT1 (StEnrolNo, StName, StAddress)

Similarly, you can use FD (2) to decompose the STUDENT relation further, but what about the attribute 'InOffice'? You find in FD (2) that Course code (CoNo) attribute uniquely determines the name of instructor (refer to FD 2(a)). Also, the FD (3) means that the name of the instructor uniquely determines the office number. This can be written as:

CoNo	→	CoInstructor	(2 (a)) (without CoName)
CoInstructor	→	InOffice	(3)

The above two FDs imply a transitive dependency:

CoNo	→	InOffice	(This is transitive dependency)
------	---	----------	---------------------------------

The revised FD (2) is:

CoNo	→	CoName, CoInstructor, InOffice	(4)
------	---	--------------------------------	-----

Use this FD to create another 2NF relation:

COU_INST (CoNo, CoName, CoInstructor, InOffice)

Now, you have the following two 2NF relations, which are obtained by decomposing the STUDENT relation:

STUDENT1 (StEnrolNo, StName, StAddress)

COU_INST (CoNo, CoName, CoInstructor, InOffice)

Are these two 2NF relations sufficient to represent the relations STUDENT? No, as there is no common attribute between the relations. Therefore, they cannot be joined together to get the data of STUDENT relation. You must have a relation that joins the two decomposed relations. This relation would have the primary key attributes of the STUDENT relation and any other attribute that is fully

functionally dependent on this primary key. However, there is no attribute in STUDENT relation that is fully dependent on the composite primary key. Thus, you will create a third relation using the composite primary key of the STUDENT relation, as shown below:

COURSE_STUDENT (StEnrolNo, CoNo)

Please note that the COURSE_STUDENT relation can be joined with STUDENT1 relation on StEnrolNo attribute and with COU_INST relation on CoNo attribute.

The relation STUDENT in 2NF form would be:

STUDENT1 (<u>StEnrolNo</u> , StName, StAddress)	2NF(a)
COU_INST (<u>CoNo</u> , CoName, CoInstructor, InOffice)	2NF(b)
COURSE_STUDENT (<u>StEnrolNo</u> , <u>CoNo</u>)	2NF(c)

5.5.3 The Third Normal Form (3NF)

Although, transforming a relation that is not in 2NF into a number of relations that are in 2NF removes many of the anomalies, it does not necessarily remove all anomalies. Thus, further Normalisation is sometimes needed to ensure further removal of anomalies. These anomalies arise because a 2NF relation may have attributes that are not directly related to the primary key of the relation.

Definition: A relation is in third normal form if it is in 2NF and every non-key attribute of the relation is non-transitively dependent on the primary key of the relation.

But what is **non-transitive** dependence?

Let A, B and C be three attributes of a relation R such that $A \rightarrow B$ and $B \rightarrow C$. From these FDs, we may derive $A \rightarrow C$. This dependence $A \rightarrow C$ is transitive.

Now, let us reconsider the relation 2NF (b)

COU_INST (CoNo, CoName, Instruction, InOffice)

Assume that CoName is not unique and therefore CoNo is the only candidate key. The following functional dependencies exist:

CoNo	$\rightarrow$	CoInstructor	(2 (a))
CoInstructor	$\rightarrow$	InOffice	(3)
CoNo	$\rightarrow$	InOffice	(This is transitive dependency)

You had derived $CoNo \rightarrow InOffice$ from the functional dependencies 2(a) and (3) for decomposition to 2NF. The relation is, however, not in 3NF since the attribute 'InOffice' is not directly dependent on attribute 'CoNo' but is transitively dependent on it and should, therefore, be decomposed as it has all the anomalies. The primary difficulty in the relation above is that an instructor might be responsible for several subjects, requiring one tuple for each course. Therefore, his/her office number will be repeated in each tuple. This leads to all the problems such as update, insertion, and deletion anomalies. To overcome these difficulties, you need to decompose the relation 2NF(b) into the following two relations:

COURSE (CoNo, CoName, CoInstructor)
INST (CoInstructor, InOffice)

Please note these two relations and 2NF (a) and 2NF (c) are already in 3NF. Thus, the relation STUDENT in 3 NF would be:

STUDENT1 (StEnrolNo, StName, StAddress)
 COURSE (CoNo, CoName, CoInstructor)
 INST (CoInstructor, InOffice)
 COURSE_STUDENT (StEnrolNo, CoNo)

The 3NF is usually quite adequate for most relational database designs. There are, however, some situations where a relation may be in 3 NF but have anomalies. For example, consider a relation STUDENT5 (StEnrolNo, StName, CoNo, CoName). Assume it has the following set of FDs:

StEnrolNo	→	StName
StName	→	StEnrolNo
CoNo	→	CoName
CoName	→	CoNo

Is the relation STUDENT5 is in 3NF? The FDs of this relation can be written as:

StEnrolNo, CoNo	→	StName, CoName
StEnrolNo, CoName	→	StName, CoNo
StName, CoNo	→	StEnrolNo, CoName
StName, CoName	→	StEnrolNo, CoNo

Therefore, the relation STUDENT5 has the following candidate keys.

(StEnrolNo, CoNo)
 (StEnrolNo, CoName)
 (StName, CoNo)
 (StName, CoName)

Figure 5.6 shows the functional dependency diagram for this relation.

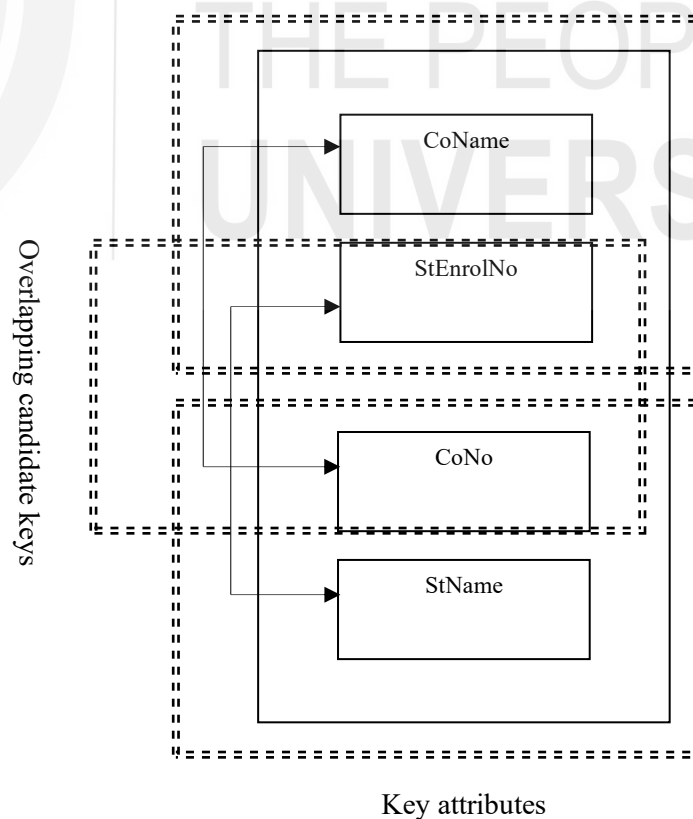


Figure 5.6: Functional Diagram for STUDENT5 relation

You may observe that all the attributes of STUDENT5 relations are prime attributes as they are part of some candidate key. You may also observe that the relation has overlapping candidate keys. Therefore, no non-key attribute exists in the relation. Hence, the relation follows the criteria of 2NF and 3NF and is in 3NF. However, this relation still suffers from the three anomalies (please make an instance for the relation), as the attributes in the non-overlapping part of the candidate key can determine each other. Therefore, the relation is to be normalised into a stronger normal form called BCNF.

5.5.4 Boyce-Codd Normal Form (BCNF)

As stated earlier, the relation STUDENT5 (StEnrolNo, StName, CoNo, CoName) has the following candidate keys:

(StEnrolNo, CoNo) ; (StEnrolNo, CoName)
(StName, CoNo) ; (StName, CoName)

Since the relation has no non-key attributes, the relation is in 2NF and also in 3NF. However, the relation suffers from anomalies (please check it yourself by making the relational instance of the STUDENT5 relation).

The difficulty in this relation is being caused by dependence within the attributes of composite candidate keys.

Definition: A relation is in Boyce-Codd Normal Form (BCNF) if it fulfills the following criteria:

- (i) The relation fulfills the criteria of the Third Normal Form (3NF), and
- (ii) All the determinants (the left side of an FD) are the candidate key.

A relation that is in 3NF and does not have overlapping candidate keys, will also be in BCNF. However, if a 3NF relation have following features then it is NOT in BCNF:

- (a) It has overlapping composite candidate keys.
- (b) The non-overlapping attributes of two candidate keys are functionally dependent.

For example, in the relation STUDENT5, the candidate keys:

(StEnrolNo, CoName) and (StName, CoName) have non-overlapping attributes StEnrolNo and StName. Further, $\text{StEnrolNo} \rightarrow \text{StName}$

The relation STUDENT5 (StEnrolNo, StName, CoNo, CoName) is in 3NF, but not in BCNF.

You can decompose the relation into the following relations using FD $\text{StEnrolNo} \rightarrow \text{StName}$ as:

STUDENT6(StEnrolNo, CoNo, CoName)

STUDENTNAME (StEnrolNo, StName)

STUDENT6 is still not in BCNF as it has overlapping candidate keys:

(StEnrolNo, CoNo) and (StEnrolNo, CoName) and the FD $\text{CoNo} \rightarrow \text{CoName}$

Thus, STUDENT6 can be decomposed using the FD as above to relations:

STUDENT7 (StEnrolNo, CoNo)

STUDENTNAME (StEnrolNo, StName)

COURSENAME (CoNo, CoName)

The following is another example of BCNF. Consider the relation:

ENROL (StEnrolNo, StName, CoNo, CoName, Dateenrolled)

Let us assume that the relation has the following FDs (We have assumed that CoName are unique):

StEnrolNo $\rightarrow$ StName
CoNo $\rightarrow$ CoName

CoName → CoNo
StEnrolNo, CoNo → Dateenrolled

The candidate keys of the relation would be:

(StEnrolNo, CoNo)
(StEnrolNo, CoName)

Due to dependency between the non-overlapping attributes of the candidate keys, the 3NF relation ENROL is NOT in BCNF. This relation will have all the stated anomalies (Please draw the relational instance and check these problems). The BCNF decomposition of the relation would be:

STUD1 (StEnrolNo, StName)
COU1 (CoNo, CoName)
ENROL1 (StEnrolNo, CoNo, Dateenrolled)

You now have a relation that only has information about students, another only about subjects and the third only about relationship enrolls.

Higher Normal Forms:

Are there more normal forms beyond BCNF? Yes, however, these normal forms are not based on the concept of functional dependence. Further normalisation is needed if the relation has multi-valued, join dependencies, etc. These are discussed in Unit 6.

Check Your Progress 2

- 1) Given the following relation of book issue and return in a Library:
BOOKISSUERETURN (student_id, student_name, bookID, book_title, issuedOn, returnedOn)
A student can get many books issued to him/her. Explain anomalies in the relation above with the help of a relational instance.
.....
.....
- 2) Identify the functional dependencies in the relation given in question 1. What are the candidate keys and which of these can be a primary key?
.....
.....
- 3) Normalise the relation of problem 1.
.....
.....
.....
.....

5.6 DESIRABLE PROPERTIES OF DECOMPOSITION

In the previous section, you have learnt the process of normalisation using functional dependency. Normalizing a relation entails decomposing a relation into a number of non-disjoint projections of the relation based on the FDs, so that the three anomalies are minimised. But why are these projections non-disjoint? The non-disjoint relations allow reconstruction of the original relation instance through JOIN operation. In this section, we discuss about the properties of the a good decomposition.

The decomposition of a relation should fulfill the following:

Attribute Preservation: All the attributes of the relation, which is being decomposed, should be part of at least one of the decomposed relations. This is a necessary condition, otherwise the decomposition will be lossy.

Lossless-Join Decomposition: For explaining the concept of lossless-join decomposition, let us first explain a lossy decomposition with the help of an example. This example also explains, why FDs should be used to perform proper decomposition of a relation.

Example: Consider the following instance of a STUDENT9 relation.

STUDENT9 (StEnrolNo, CoNo, Dateenrolled, CoInstructor, InRoom)

With the following set of FDs:

StEnrolNo, CoNo $\rightarrow$ Dateenrolled

CoNo $\rightarrow$ CoInstructor

CoInstructor $\rightarrow$ InRoom

Suppose you decompose the STUDENT9 relation randomly into two relations ST1 and ST2 as follows:

ST1 (StEnrolNo, CoNo, Dateenrolled, CoInstructor)

ST2 (StEnrolNo, InRoom)

Are there problems with this decomposition? Yes, but for the time being, let us focus on the question, whether this decomposition is lossless? Consider the following instance of the relation:

STUDENT9

<u>StEnrolNo</u>	CoNo	Dateenrolled	CoInstructor	InRoom
1001	MCS-201	01-02-2022	Preeti	1
1001	MCS-202	01-02-2022	Salim	2
1002	MCS-201	13-02-2022	Preeti	1
1002	MCS-203	13-02-2022	Shashi	3
1003	MCS-202	15-02-2022	Salim	2

Figure 5.7: An instance of STUDENT9 relation

The decomposed relations ST1 and ST2 would be:

ST1

<u>StEnrolNo</u>	CoNo	Dateenrolled	CoInstructor
1001	MCS-201	01-02-2022	Preeti
1001	MCS-202	01-02-2022	Salim
1002	MCS-201	13-02-2022	Preeti
1002	MCS-203	13-02-2022	Shashi
1003	MCS-202	15-02-2022	Salim

ST2

<u>StEnrolNo</u>	InRoom
1001	1
1001	2
1002	1
1002	3
1003	2

Will you be able to reconstruct the original instance of relation using ST1 and ST2? For this simply take a JOIN of the two relations ST1 and ST2 on StEnrolNo, which is the only common attribute. The joined instance would be:

ST1JOINST2

<u>StEnrolNo</u>	CoNo	Dateenrolled	CoInstructor	InRoom
1001	MCS-201	01-02-2022	Preeti	1
1001	MCS-201	01-02-2022	Preeti	2
1001	MCS-202	01-02-2022	Salim	1
1001	MCS-202	01-02-2022	Salim	2
1002	MCS-201	13-02-2022	Preeti	1
1002	MCS-201	13-02-2022	Preeti	3
1002	MCS-203	13-02-2022	Shashi	1

1002	MCS-203	13-02-2022	Shashi	3
1003	MCS-202	15-02-2022	Salim	2

Thus, the resulting relation obtained is not the same as that of *Figure 5.7*. The joined relation contains a number of spurious tuples that were not in the original relation. Because of these additional tuples, you have lost the information, such as the instructor Preeti is located in Room number 1. Such decompositions are called **lossy decompositions**. A lossless join decomposition guarantees that the join will result in exactly the same relation as was decomposed. One might think that there might be other ways of recovering the original relation from the decomposed relations but, sadly, no other operators can recover the original relation if the join does not (why?). You need to analyse why the decomposition is lossy. The common attribute in the above decompositions was StEnrolNo. The common attribute is the glue that gives us the ability to find the relationships between different relations by joining the relations together. *If the common attribute has been the primary key of at least one of the two decomposed relations, the problem of losing information would not have existed.* You may use FDs to decompose the relation STUDENT9 into 2NF and then to 3NF, to produce the following relational instance:

STENROL (StEnrolNo, CoNo, Dateenrolled)
using the FD StEnrolNo, CoNo $\rightarrow$ Dateenrolled

STENROL

StEnrolNo	CoNo	Dateenrolled
1001	MCS-201	01-02-2022
1001	MCS-202	01-02-2022
1002	MCS-201	13-02-2022
1002	MCS-203	13-02-2022
1003	MCS-202	15-02-2022

COUINST (CoNo, CoInstructor) using the FD CoNo $\rightarrow$ CoInstructor

COUINST

CoNo	CoInstructor
MCS-201	Preeti
MCS-202	Salim
MCS-203	Shashi

INSROOM (CoInstructor, InRoom) using the FD CoInstructor $\rightarrow$ InRoom

INSROOM

CoInstructor	InRoom
Preeti	1
Salim	2

You may please join the three relations to check that that the decomposition is lossless-join decomposition. *The dependency-based decomposition scheme as discussed in the section 5.5 creates lossless decomposition.*

Dependency Preservation: It is clear that the decomposition must be lossless so that you do not lose any information from the relation that is decomposed. Dependency preservation is another important requirement since a *dependency is a constraint* on the database. If all the attributes appearing on the left and the right side of a dependency appear in the same relation, then a dependency is considered to be preserved. Thus, dependency preservation can be checked easily. Dependency preservation is important, because as stated earlier, dependency is a constraint on a relation. Thus, if a constraint is split over more than one relation (dependency is not preserved), the constraint would be difficult to meet. We will not discuss this in more detail in this unit. You may refer to the further readings for more details. However, let us state some basic points:

“Normalisation to 3NF is lossless decomposition. It also preserves the dependencies.”

“Normalisation to BCNF is lossless decomposition. However, it may not preserve dependencies.”

Reduction of Redundancy: Redundancy is the cause of anomalies. Normalisation results in reduction of redundancy.

5.6 RULES OF DATA NORMALISATION

In the previous sections, you have gone through various normal forms. In this section, the normalisation using FDs is presented as a set of practical rules:

1. Identify and eliminate Attributes with multiple data values into separate relations.
2. Identify and eliminate those attributes, which are not the part of any key attributes but are dependent on the part of the composite primary key.
3. Identify and eliminate those non-key attributes, which are dependent on other non-key attributes.
4. Identify and eliminate non-overlapping composite candidate key attributes, which are dependent on each other.

Let's explain these steps of Normalisation through an example. Let us make a list of all the employees in the company. In the original employee list, each employee name is followed by any databases that the employee has experience with. Some might know many, and others might not know any.

Employee ID (empid)	Employee Name (empname)	DBMS Known (DBMS)	Department (dept)	Department Location (loc)
101	Gurmeet	MySQL	Quality Assurance	Mumbai
102	Hanif	DB2	Database Design	Delhi
103	Manish	Oracle, PostgreSQL	Frontend Design	Hyderabad
104	Sameer Singh	SQLserver, Oracle	Database Design	Delhi

Figure 5.8: Table of Data Not in 1NF

For attributes Department and Department Location will be considered in Step-3.

5.6.1 Eliminate Repeating Groups

Please observe that Figure 5.8 has a repeating group – DBMS Known. The problem with such repeating groups can be explained with the help of a query. Consider a query - “Find the list of employees, who know Oracle”.

To answer this query, you need to perform an awkward scan of the entire list of attributes Database-Known, looking for references to DB2. This is inefficient and an extremely untidy way to retrieve information.

You may convert the relation to 1NF (For the following discussion, we have ignored two attributes - Department and Department Location. These two attributes will be part of Employee relation). The two decomposed relations on eliminating the repeating groups are shown in Figure 5.9. The elimination of repeating group DBMS Known from the Employee relation, which has primary key - Employee ID, leaves the Employee relation in 1NF. However, this elimination requires that you add another relation, which can store information about the DBMS known by each employee. This new relation is named DBMSknown. It has three attributes, including a foreign key for relating the two relations with a JOIN operation. Now, you can answer the question by looking in the database relation for "Oracle" and getting the list of Employees. Please note that although the name of the DBMSs are unique, yet we have added a DBMS code field in the decomposed relation (Figure 5.8):

EMPLOYEE			
<u>empid</u>	empname	dept	loc
101	Gurmeet	Quality Assurance	Mumbai
102	Hanif	Database Design	Delhi
103	Manish	Frontend Design	Hyderabad
104	Sameer Singh	Database Design	Delhi

DBMSknown		
<u>empid</u>	DBMScode	DBMS
101	1	MySQL
102	2	DB2
103	3	Oracle
103	4	PostgreSQL
104	5	SQLserver
104	3	Oracle

Figure 5.9: 1NF relation after Eliminating Repeating Groups

5.6.2 Eliminate Redundant Data

In the “DBMSknown” relation above, the primary key is made up of the *empid* and the *DBMScode*. The attribute *DBMS* depends only on the *DBMScode*. The same *DBMS* will appear redundantly every time its associated *DBMScode* appears in the *DBMSknown* Relation. Thus, this database relation has redundancy, for example, *DBMScode* value 3 is Oracle, which is repeated twice. In addition, it also suffers insertion anomaly that is you cannot enter Sybase in the relation as no employee has that database skill.

The deletion anomaly also exists. For example, if you remove an employee with *empid* 3; no employee has a skill in PostgreSQL and the information that *DBMScode* 4 is the code of PostgreSQL will be lost.

To avoid these problems, you need a second normal form. To achieve this, you isolate the attributes that depend on the entire composite key from the attributes that depend only on the *DBMScode*. This results in decomposition of *DBMSknown* relation into two relations: “DBMSlist” and “EMPDBMS” which lists the databases for each employee. The EMPLOYEE relation is already in 2NF as all the *empid* determines all other attributes.

EMPDBMS	
<u>empid</u>	<u>DBMScode</u>
101	1
102	2

DBMSlist	
<u>DBMScode</u>	DBMS
1	MySQL
2	DB2

103	3
103	4
104	5
104	3

3	Oracle
4	PostgreSQL
5	SQLserver

5.6.3 Eliminate Columns Not Dependent on Key

The Employee Relation satisfies -

First normal form - As it contains no repeating groups.

Second normal form - As it does not have a multi-attribute key.

Now, let us add the remaining two attributes of Employee relation. The employee relation is in 2NF but not 3NF. Why?

EMPLOYEE			
<u>empid</u>	empname	dept	loc
101	Gurmeet	Quality Assurance	Mumbai
102	Hanif	Database Design	Delhi
103	Manish	Frontend Design	Hyderabad
104	Sameer Singh	Database Design	Delhi

The key to the Employee relation is *empid*. The attribute *loc* describes information about the Department and not about an Employee. To achieve the third normal form, *dept* and *loc* must be moved into a separate relation. Since they describe a department, thus, the attribute *dept* becomes the key of the new “Department” relation. The motivation for this decomposition is that you want to avoid update, insertion and deletion anomalies.

EMPLOYEElist		
<u>empid</u>	empname	dept
101	Gurmeet	Quality Assurance
102	Hanif	Database Design
103	Manish	Frontend Design
104	Sameer Singh	Database Design

DEPARTMENT Relation		
<u>deptid</u>	dept	loc
1	Quality Assurance	Mumbai
2	Database Design	Delhi
3	Frontend Design	Hyderabad

You may use these steps for decomposition till BCNF.

Check Your Progress 3

- Why should functional dependencies be preserved in decomposition?
.....
.....
.....
.....
.....
- Explain the term lossless-join decomposition.
.....
.....
.....
.....
- List various steps that may be followed to decompose a relation up to BCNF.

.....

.....

.....

.....

5.7 SUMMARY

This unit discusses some of the most important aspects of a good database design. The unit first defines the concept of referential integrity constraint and entity integrity constraints. It then discusses different forms of normalisation based on the concept of function dependency. A functional dependency highlights the relationships among the attributes of a relation. Different dependency among attributes leads to the problems of update anomalies, insertion anomalies and deletion anomalies. Different forms of normalisation decompose relations, using the FDs, to remove anomalies. The concept of FD is used for the process of normalisation.

This unit also defines the features of a good decomposition of a relation. The most important being attribute preservation, dependency preservation and lossless-join decomposition. Finally, the unit summarises the rules of normalisation with the help of an example.

5.10 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) The following table shows the candidate keys and primary keys of the relations:

Relation	Candidate Keys	Primary key
S (Supplier)	SNO, SNAME*	SNO
P (Part)	PNO, PNAME*	PNO
J (Project)	PROJ NO, JNAME*	PROJNO
SPJ	(SNO+PNO+PROJNO)	(SNO+PNO+PROJNO)

* Only if the values are assumed to be unique, this may be incorrect for large systems.

- 2) SNO in S, PNO in P, PROJNO in J and (SNO+PNO+PROJNO) in SPJ should not contain NULL value. Also, no part of the primary key of SPJ, that is SNO or PNO or PROJNO should contain NULL value.
- 3) Foreign keys exist only in SPJ, where SNO references SNO of S; PNO references PNO of P; and PROJNO references PROJNO of J. The referential integrity necessitates that all matching foreign key values must exist.
- 4) You may select the following referential actions:
For operations like Delete and update, you should use referential action as RESTRICT. This selection would restrict loss of information from the SPJ relation, in case you delete a tuple in S or P or J relation. Insertion of records does not create a problem provided the referential integrity constraints are met.

Check Your Progress 2

- 1) The database suffers from all the anomalies; let us demonstrate these with the help of the following relation instance or state of the relation:

student_id	Student_name	bookID	Book_title	issuedOn	returnedOn
A 101	Abishek	0050	DBMS	15/01/05	25/01/05
R 102	Raman	0125	DS	25/01/05	29/01/05
A 101	Abishek	0060	Multimedia	20/01/05	NULL
R 102	Raman	0050	DBMS	28/01/05	NULL

Is there any data redundancy?

Yes, the information is getting repeated about student names and book Title. This may lead to an update anomaly in case of changes made to the data value of the book. In addition, the library may be having many more books that have not been issued yet. The information of such books cannot be added to the relation as the primary key to the relation is: (student_id + bookID + issuedOn). (This would involve the assumption that the same book can be issued to the same student only once in a day). Thus, you cannot enter the information about bookID and book_title of those books, which has not been issued to a student. This is an insertion anomaly. Similarly, you cannot enter student_id if a book is not issued to that student. This is also an insertion anomaly. As far as the deletion anomaly is concerned, suppose Abishek did not collect the Multimedia book, so this record needs to be deleted from the relation (tuple 3). This deletion will also remove the information about the Multimedia book that is its bookID and book_title. This is a deletion anomaly for the given instance.

- 2) The FDs of the relation are:
- student_id $\rightarrow$ student_name (1)
 - bookID $\rightarrow$ book_title (2)
 - bookID, student_id, issuedOn $\rightarrow$ returnedOn (3)

Why is the attribute issuedOn on the left hand of the FD above? Because a student, for example, Abishek can be issued the book having the bookID 0050 again on a later date, let say in February after Raman has returned it. Also note that returnedOn may have NULL value, but that is allowed in the FD. FD only necessitates that the value may be NULL or any specific data that is consistent across the instance of the relation.

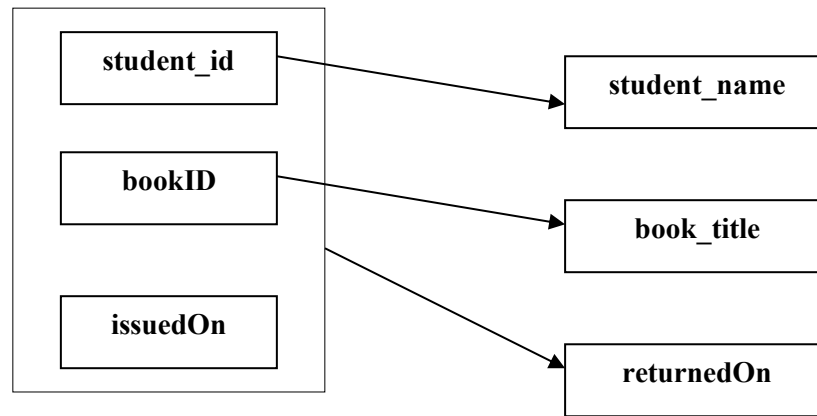
Just one candidate key, which is also chosen as the primary key, is: bookID, student_id, issuedOn.

Some interesting domain and procedural constraints in this database are:

- A book cannot be issued again unless it is returned.
- A book can be returned only after the date on which it has been issued.
- A student can be issued a limited/maximum number of books at a time.

You will study about these constraints later in the Block.

- 3) The relation is in 1NF. The following is a FD diagram for the relation:



1NF to 2NF:

The composite key to the relation is (student_id + bookID). The attributes student_name depends on student_id, which is part of the composite key, likewise book_title attributes is dependent on bookID attribute, which is part of the composite key so the decomposition based on following FDs would be:

MEMBER (student_id, student_name) [Reason: FD (1)]
 BOOK (bookID, book_name) [Reason: FD (2)]
 ISSUE_RETURN (student_id, bookID, issuedOn, returnedOn) [Reason: FD (3)]

2NF to 3 NF and BCNF:

All the relations are in 3NF and BCNF also. As there is no dependence among non-key attributes and there are no overlapping candidate keys.

Please note that the decomposed relations have no anomalies. Let us map the relation instance here:

MEMBER

Member - ID	Members Name
A 101	Abhishek
R 102	Raman

BOOK

Book - Code	Book – Name
0050	DBMS
0060	Multimedia
0125	OS

ISSUE_RETURN

Member - ID	Book - Code	Issue - Date	Return – Date
A 101	0050	15/01/05	25/01/05
R 102	0125	25/01/05	29/01/05
A 101	0060	20/01/05	NULL
R 102	0050	28/01/05	NULL

- There is no redundancy, so no update anomaly is possible in the relations above.
- The insertion of new book and new student can be done in the BOOK and MEMBER tables respectively without any issue of book to student or vice versa. So, no insertion anomaly.
- Even on deleting record 3 from ISSUE_RETURN it will not delete the information the book 0060 titled as Multimedia as this information is in separate table. So, no deletion anomaly.

Check Your Progress 3

- 1) Dependency preservation within a relation helps in enforcing constraints that are implied by dependency over a single relation. In case, you do not preserve dependency then constraints might be enforced on more than one relation that is quite troublesome and time consuming.
- 2) A lossless join decomposition is one in which you can reconstruct the original table without loss of information that means exactly the same tuples are obtained on taking join of the relations obtained on decomposition. Lossless join decomposition requires that decomposition should be carried out on the basis of FDs.
- 3) The steps of Normalisation are:
 1. Remove repeating groups of each of the multi-valued attributes.
 2. Then remove redundant data and its dependence on part keys.
 3. Remove columns from the table that are not dependent on the key, that is remove transitive dependency.
 4. Check if there are overlapping composite candidate keys. If yes, check for any dependency among the non-overlapping attributes of the composite candidate keys; and remove such dependency.

